

Winning Idea Dossier

GAVEL: Guarantee-Aware Validation with Executable Logic

Final score: 92

Recommended venue: ICLR

Venue rationale: ICLR is the strongest fit because PACT-EL is a technical method for adapting prompts and compound LM systems, with a clear learning-systems contribution and direct relevance to modern prompt/program optimizers such as MIPROv2 and GEPA. It is not broad enough for Nature, but it is squarely in scope for a top ML venue if the matched-budget evidence is strong.

Why This Idea Wins

Selected as the winner after rebuilding raw GAVEL into PACT-EL: it changes ultra-frugal prompt optimization from candidate search into typed behavioral-contract repair plus canary falsification, giving the round a genuinely new optimized object and update rule.

Problem

Natural-language clauses are vague, so optimizers over-fit text while under-enforcing hard guarantees like formatting and safety.

Broken Assumption

That guarantees must remain written in prose inside the prompt.

Paradigm Shift

Extract guarantees into a compact, executable policy language embedded as comments, let the interpreter enforce them, and keep the prompt for fuzzier guidance only.

Why This Is Not Incremental

Describable to a layperson as 'turn the rules into mini-code the model must follow rather than more words'—no link to any existing search framework required.

Core Method

One optimizer call compiles prose constraints into a JSONLogic-style program, deletes redundant prose, appends an inline interpreter call instruction, and tests with four synthesized inputs that intentionally try to violate each guarantee branch.

Novelty Claim

Transforms parts of the prompt into a runnable logic payload that the model must interpret, creating a hybrid code-prompt artifact unseen in prior pure-text optimizers.

Why Now

Models have grown adept at following inline pseudo-code; encoding guarantees programmatically shrinks ambiguity exactly when compliance stakes are rising.

Python Feasibility Snapshot

Generate JSONLogic with a single LLM call, embed it, implement canaries as unit tests, run them via asyncio; entire flow is straightforward Python scripting.

Detailed Analysis

Method name: PACT-EL, Prompt Axiom Compiler with Executable Logic. Abstract: PACT-EL optimizes a prompt by compiling intended behavior into a typed Prompt Axiom Graph, ranking defects from an Evidence Ledger, applying one minimal AxiomPatch, and spending a tiny canary budget trying to disprove the patch. Core thesis: with only 1 to 15 optimization calls, candidate search is the wrong shape because evaluation noise and tiny validation sets dominate; the better frame is to infer the broken behavioral contract, repair the faulty axiom once, and test only the causal claim behind that repair. New scientific objects: the Evidence Ledger is a structured table of observed behavior with fields such as `evidence_id`, `source`, `expected_behavior`, `observed_behavior`, `success_or_failure`, `severity`, `suspected_contract_area`, `confidence`, and `prompt_fixability`. The Prompt Axiom Graph is a typed behavior contract, not a prompt outline, with node types including `TaskIntent`, `OutputSchema`, `DecisionRule`, `ReasoningPolicy`, `ToolContract`, `RetrievalContract`, `SafetyBoundary`, `UncertaintyPolicy`, `FormattingRule`, `PrecedenceRule`, `DemonstrationRole`, `AntiPattern`, and `ModelCeilingNotice`, and edge types including `supports`, `depends_on`, `overrides`, `contradicts`, `weakens`, `duplicates`, and `risks_regression_in`. The defect posterior ranks prompt-fixable contract failures by confidence, severity, expected gain, regression risk, and prompt-fixability. The AxiomPatch is the update object: graph edits, rendered prompt diff, clauses added/removed/merged, `token_delta`, expected fixed behaviors, expected unchanged behaviors, regression risks, canaries, and rollback rule. The Executable GuaranteeScript is a JSONLogic-like local predicate language used for hard clauses and deterministic validators, but it is explicitly not advertised as a true runtime enforcement guarantee inside the model. Algorithm: build or ingest the Evidence Ledger; if no behavior evidence exists, run four cold-start probes; make one optimizer call that emits the Prompt Axiom Graph, defect posterior, one AxiomPatch, canary set, and rendered prompt under a strict schema; render the final prompt from graph nodes and precedence edges rather than freeform rewriting; run four canaries consisting of a fix canary, boundary canary, regression canary, and format/schema canary; accept only if deterministic validators, task labels, or counted judges support the patch and no critical regression appears; if falsified, perform one targeted repair that edits only the falsified axiom or edge and rerun the failed canary plus one regression canary; if still falsified, return the original prompt with a no-patch diagnosis. Pseudocode: `def pact_optimize(prompt, task_spec, rubric, logs=None, examples=None, budget=9): ledger = build_evidence_ledger(...); if not ledger.has_behavior_evidence(): probes = choose_four_probe_inputs(...); ledger.add(run_target(prompt, probes)); analysis = optimizer_compile_contract_and_patch(prompt, task_spec, rubric, ledger); results = run_target(analysis.rendered_prompt, analysis.canaries); if passes_falsification_gate(results, analysis): return analysis.rendered_prompt, analysis.report; repair = optimizer_repair_falsified_axiom(analysis,`

results); retest = run_target(repair.rendered_prompt, repair.retest_canaries); if passes_falsification_gate(retest, repair): return repair.rendered_prompt, repair.report; return prompt, make_no_patch_report(...). API-call ledger: logged mode is 1 optimizer call plus 4 target canary calls, or only 1 optimizer call if cached outputs can replay canaries; cold start is 4 target probe calls plus 1 optimizer call plus 4 target canary calls, total 9; one-repair mode is 9 plus 1 repair optimizer call plus 2 target retest calls, total 12 without judges; hard cap is 15 when up to 3 counted judge calls are unavoidable. Innovation assessment: PACT-EL passes the paradigm-shift test because it reframes prompt optimization from finding a better prompt to repairing and falsifying a behavioral contract. It passes the first-principles test because its core claim is explainable directly: if a prompt fails because its behavior contract is ambiguous, contradictory, or underspecified, represent that contract explicitly, repair the faulty axiom, and test the causal claim. Against MIPROv2, it avoids instruction/demo candidate search and Bayesian trials, while supporting multi-module systems through ToolContract, RetrievalContract, and module-specific axiom nodes. Against GEPA, it uses traces as evidence for contract defects rather than as material for reflective prompt evolution and Pareto frontier search. Against TextGrad and ProTeGi, it is more than critique-to-rewrite because the update is a typed defect hypothesis plus falsifiable patch claim. Against OPRO, PromptBreeder, and EvoPrompt, it avoids iterative candidate generation, mutation, and population search. Against Bayesian, Hyperband, and bandit selection methods, it improves sample efficiency by eliminating prompt-arm selection rather than scheduling it better. Against reflective diagnostic prompt tuning, it keeps rich feedback but replaces full-set iteration with a causal canary gate. Closest adjacent work includes LMQL, DSPy Assertions, constrained decoding, guardrail DSLs, MIPROv2, GEPA, TextGrad, ProTeGi, OPRO, and PromptBreeder; PACT-EL is distinct because it remains black-box and API-only, prompt-first, tiny-budget, auditable, and centered on one contract patch rather than many prompt candidates. The reviewer-convincing evidence would be matched-budget wins at 1, 5, 9, and 14 calls, large ablation drops when removing the graph or canary gate, low regression rates, controlled prompt-length growth, and accurate no-patch diagnoses for retrieval, tool, data, model-knowledge, and impossible-constraint failures.

Python Implementation Blueprint

Implement a Python package pact_el. Repository structure: pact_el/schemas.py for Pydantic models; ledger.py for EvidenceLedger builders; graph.py for a NetworkX PromptAxiomGraph; compiler.py for optimizer prompts, schema enforcement, and JSON parsing; renderer.py for converting graph nodes, precedence edges, and GuaranteeScript clauses into the final prompt; validators.py using jsonschema, regex, exact match, enum checks, numeric predicates, and jsonlogic-style evaluation; canaries.py for fix, boundary, regression, and format/schema canary construction; clients.py with async target and optimizer clients via LiteLLM or provider SDKs; falsification.py for acceptance and rollback logic; experiments/run.py for matched-budget baselines and ablations. Data flow: ingest logs/examples, normalize to EvidenceRow objects, compile graph and patch with one optimizer call, render the patched graph, run target canaries asynchronously, validate locally, optionally run one targeted repair, then write an OptimizationReport with full call ledger, patch diff, validator outputs, and no-patch diagnosis if needed.

Implementation Milestones

- Define strict Pydantic schemas for EvidenceRow, AxiomNode, AxiomEdge, PromptAxiomGraph, DefectHypothesis, AxiomPatch, GuaranteeScript, Canary, CanaryResult, CallLedger, and OptimizationReport, with no silent LLM JSON-fix retries.
- Build Evidence Ledger ingestion from cached logs, examples, grader outputs, human critiques, agent traces, tool outputs, and optional four-probe cold-start runs, including severity, confidence,

suspected_contract_area, and prompt_fixability fields.

- Create the single optimizer-compiler prompt that outputs the Prompt Axiom Graph, defect posterior, one AxiomPatch, four required canaries, rollback rule, and rendered prompt under a strict JSON schema.
- Implement the graph renderer so the final prompt is produced from graph nodes, precedence edges, and executable guarantee clauses rather than from a freeform rewrite.
- Implement deterministic canary validators using jsonschema, regex, exact match, enum checks, numeric checks, and JSONLogic-like local predicates; add a counted LLM-judge fallback only where deterministic validation is impossible.
- Implement the falsification gate: no critical regression, fix claim supported or correctly diagnosed as non-prompt-fixable, output contract preserved, token_delta bounded, and no task-semantic drift.
- Add the one-repair rule that edits only the falsified axiom or edge, reruns the failed canary plus one regression canary, and returns a no-patch diagnosis after a second failure.
- Package reproducible experiment runners with cached API responses, async target and optimizer clients, seed control, token/cost accounting, ablation toggles, and automatic call-ledger reports.

Benchmark Plan

- IFEval and IFBench-style verifiable instruction following: compare original prompt, manual cleanup, one-shot LLM rewrite, one-shot OPRO, one-step TextGrad/ProTeGi-style critique rewrite, GEPA matched to 1/5/9/14 calls, and MIPROv2 matched to the same budgets. Metrics: strict pass rate, loose pass rate, optimization calls, target calls, token delta, prompt length, and regression rate. Success signal: best or tied-best pass rate under 5 or 9 calls with materially fewer regressions than broad rewrites.
- JSONSchemaBench-inspired structured-output and extraction tasks: compare PACT-EL against prose constraints, provider JSON mode or structured-output references where available, one-shot rewrite, matched-budget GEPA, and matched-budget MIPROv2. Metrics: schema validity, semantic F1, malformed-output rate, target calls, judge calls, and cost per valid output. Success signal: lower violation rate without sacrificing semantic accuracy, especially when the graph renderer makes schema clauses explicit.
- GSM8K and BBH reasoning-prompt repair: compare original prompt, human cleanup, one-shot critique rewrite, matched-budget GEPA, matched-budget MIPROv2, and normal-budget GEPA/MIPROv2 reference ceilings. Metrics: exact-match accuracy, cost per accuracy point, variance across seeds, and regression on easy cases. Success signal: clear matched-budget gain at 9 or 14 calls, while honestly acknowledging that full-budget search may remain a higher ceiling on some reasoning tasks.
- HotpotQA or comparable RAG prompt-and-boundary suite: compare PACT-EL with reflective diagnostic prompt tuning, GEPA, MIPROv2, and a retrieval/config optimizer boundary reference. Metrics: answer F1, supporting-fact F1, prompt-only ceiling detection accuracy, and harmful-patch rate when evidence is missing. Success signal: improvement on prompt-fixable failures and accurate no-patch labels for retrieval gaps.
- Tau-bench or similar tool-agent policy tasks: compare PACT-EL against GEPA, MIPROv2, reflective prompt tuning, and graph/config optimizers as boundary references. Metrics: task success, policy-violation rate, tool-call correctness, pass^k stability, and optimization calls. Success signal: fewer tool-contract and policy regressions under 14 calls, plus honest escalation when tool or data changes are required.

- Mechanism ablations across all suites: remove the Prompt Axiom Graph, remove the Evidence Ledger, remove the defect posterior, remove the minimality penalty, choose random canaries, allow broad rewrite instead of AxiomPatch, remove prompt-only ceiling labels, and allow two-prompt empirical selection. Metrics: task quality, regression rate, token growth, canary predictive value, and cost per quality point. Success signal: statistically meaningful drops when each distinctive component is removed.
- Decisive novelty test: Constraint compliance test: On tasks with strict output format or safety rules, compare GAVEL's logic-enhanced prompt vs. a baseline prompt with plain-text rules and vs. an external grammar-constrained method. Measure the rate of rule violations (format errors, forbidden content) and task success to confirm GAVEL yields higher compliance under the same low-call budget.
- Decisive novelty test: Ablation study: Use the same prompt optimization pipeline but without converting rules to logic (keeping all instructions as normal text). Evaluate outputs to see if the logic embedding provides a statistically significant improvement in adherence to guarantees and output correctness, isolating the benefit of the executable logic component.

Road To State Of The Art

To push PACT-EL to state of the art under tiny budgets, scale breadth rather than loop depth: build a reusable library of axiom-node templates, local validators, failure taxonomies, and no-patch diagnoses across many tasks while keeping per-task optimization to one compile call and one optional repair. Report normal-budget GEPA and MIPROv2 as reference ceilings, but make the primary claim matched-budget superiority at 1, 5, 9, and 14 calls. Prioritize settings where logs reveal repeated contract defects: schema drift, precedence conflicts, refusal-boundary ambiguity, retrieval-use policy errors, tool-contract violations, and format regressions. Add diagnostics that separate prompt-fixable ambiguity from missing knowledge, bad retrieval, invalid upstream data, broken tools, and impossible output contracts. The decisive SOTA signature is not universal accuracy dominance; it is higher quality per optimization call, lower regression rate, smaller token growth, stronger canary-to-held-out predictiveness, and better prompt-only ceiling detection than search-based optimizers.

Key Risks To Retire Early

- Empirical risk: some target models may treat inline GuaranteeScript as decorative text, so the paper must show that typed executable clauses plus rendering and local validation outperform equivalent prose under matched budgets.
- Compiler risk: the optimizer LM can produce a plausible but wrong graph or defect posterior; strict schemas, deterministic validators, and graph/ledger ablations are needed to show this remains controlled.
- Canary risk: four canaries may be too narrow and could accept a patch that passes canaries while changing task semantics; held-out evaluation and semantic-drift checks are mandatory.
- Scope risk: PACT-EL must not claim to fix missing knowledge, retrieval gaps, broken tools, invalid data, or impossible constraints; its credibility depends on returning no-patch diagnoses for these cases.

Supporting Sources

- Prompting Is Programming: A Query Language for Large Language Models:
<https://arxiv.org/abs/2212.06094>

- SPML: A DSL for Defending Language Models Against Prompt Attacks:
<https://arxiv.org/abs/2402.11755>